

IMPERIAL

Finding and Understanding Bugs in ZKP Compilers

JMC MEng FYP

Jakub Cabala
June, 2026

Acknowledgements



Prof. Alastair Donaldson



Dr. Stefanos Chaliasos

Overview

01

Zero-Knowledge Proofs Background

What ZKPs are, and how proofs are generated and verified

02

Current Landscape & Enhancements

Existing testing tools, and my improvements to Circuzz

03

SMT-Based Framework

My SMT-driven approach to testing ZKP compilers

04

Results & Evaluation

How did the framework perform

05

Conclusion

Summary and further topics

01

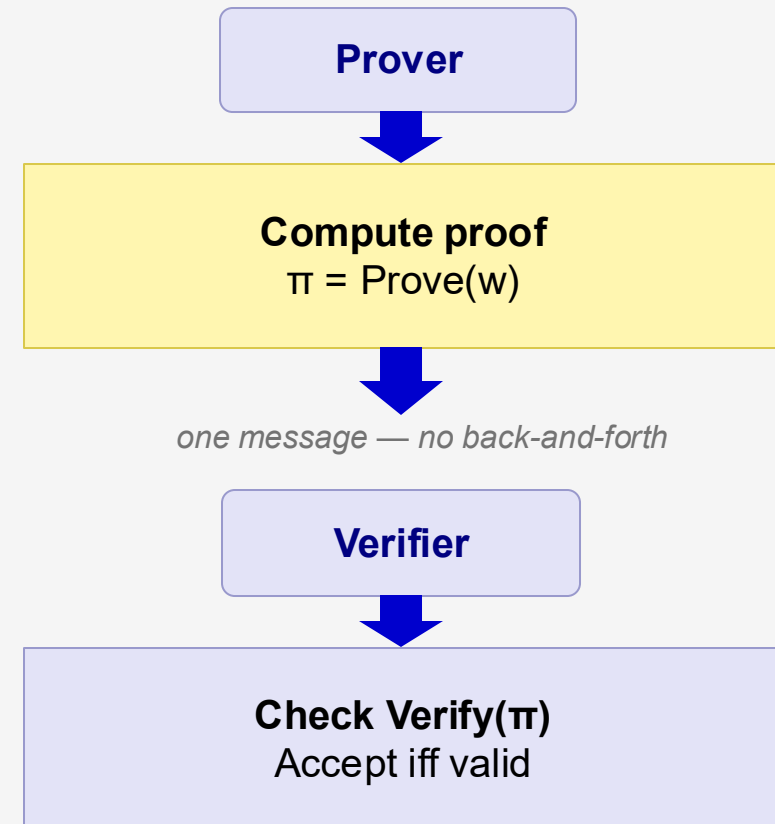
Zero-Knowledge Proofs Background

A ZKP is a cryptographic protocol that allows a prover to convince a verifier that a statement is true without revealing any additional information.

Simple ZKPs

ZKPs are fundamentally protocols (sets of rules to follow) between a prover and a verifier, but a useful mental model is to view them as compressed into two algorithms **Prove** and **Verify** .

Classical examples include proving knowledge of a **Hamiltonian cycle** in a graph or a **discrete logarithm**

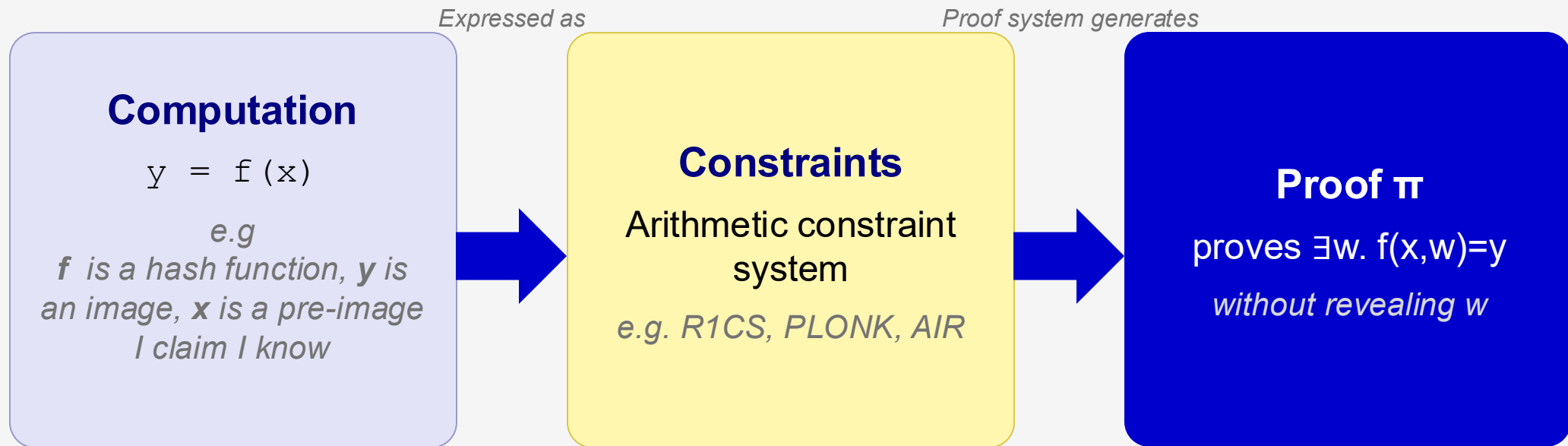


Key Properties

Property	Meaning
Completeness	Valid proofs are accepted by the verifier
Soundness	Invalid statements cannot be proven (except with negligible probability)
Zero-Knowledge	No information is revealed beyond the truth of the statement

Modern ZKPs – From Computation to Proof

Modern ZKPs prove knowledge of values that satisfy a computation, **for any computation expressible as a constraint system.**



Modern proof systems, e.g Groth16 [2], PLONK [3], STARKs [4], work for any computation expressed this way

[2] Groth J. On the Size of Pairing-Based Non-Interactive Arguments. In: Advances in Cryptology – EUROCRYPT 2016. vol. 9666 of Lecture Notes in Computer Science. Springer; 2016. p. 305-26. Available from: https://doi.org/10.1007/978-3-662-49896-5_11.

[3] Gabizon A, Williamson ZJ, Ciobotaru O. PLONK: Permutations over Lagrange-Bases for Oecumenical Noninteractive Arguments of Knowledge; 2019. Accessed: 2026-01-18. Cryptology ePrint Archive, Report 2019/953. Available from: <https://eprint.iacr.org/2019/953>.

[4] Ben-Sasson E, Bentov I, Horesh Y, Riabzev M. Scalable, Transparent, and Post-Quantum Secure Computational Integrity; 2018. Accessed: 2026-01-18. Cryptology ePrint Archive, Report 2018/046. Available from: <https://eprint.iacr.org/2018/046>.

R1CS – Rank-1 Constraint Systems

R1CS is the most common constraints system [2]

General Form

$$\langle a_i, z \rangle \cdot \langle b_i, z \rangle = \langle c_i, z \rangle$$

z = all variables

*each constraint: a product of two linear combinations
equals a third*

Example

$$y = x^2 + 3$$

↓ *R1CS constraints*

$$x \cdot x = t$$

$$t + 3 = y$$

$$z = (1, x, t, y)$$

Non-quadratic expressions must be decomposed into a sequence of these quadratic constraints

[2] Groth J. On the Size of Pairing-Based Non-Interactive Arguments. In: Advances in Cryptology – EUROCRYPT 2016. vol. 9666 of Lecture Notes in Computer Science. Springer; 2016. p. 305-26. Available from: https://doi.org/10.1007/978-3-662-49896-5_11.

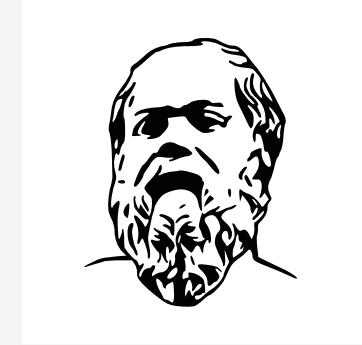
ZKP DSLs & Compilers

A compiler's job:

- **Let you write** computations in a higher-level DSL, rather than authoring constraints directly
- **Optimise** the generated constraints, e.g. removing redundant ones

DSLs covered:

- Circom [15] — circuit DSL targeting R1CS / Groth16
- ZoKrates [7] — Ethereum-oriented zkSNARK toolbox
- Gnark [17] — Go-based framework, native Go syntax
- Noir [16] — higher-level DSL, compiles to ACIR [19]



[7] Eberhardt J, Tai F. ZoKrates: A Toolbox for zkSNARKs on Ethereum; 2018. Accessed: 2026-01-13. Available from: <https://zokrates.github.io/>.

[15] Bellés-Muñoz M, Isabel M, Muñoz-Tapia JL, Rubio A, Baylina Melé J. Circom: A Circuit Description Language for Building Zero-Knowledge Applications. IEEE Transactions on Dependable and Secure Computing. 2023;20(6):4733-51. Available from: <https://doi.org/10.1109/TDSC.2022.3232813>.

[16] Noir Team. Noir;. Accessed: 2026-01-18. Available from: <https://noir-lang.org>.

[17] ConsenSys. gnark Documentation;. Accessed: 2026-01-18. Available from: <https://docs.gnark.consensys.io>.

[19] Aztec Protocol. ACIR: Abstract Circuit Intermediate Representation;. Software repository. Accessed: 2026-05-22. <https://github.com/noir-lang/acvm>.

Image sources — Circom: docs.circom.io · Gnark: github.com/ConsenSys/gnark · Noir: noir-lang.org · ZoKrates: <https://github.com/zokrates/zokrates>

Practical Applications of ZKPs

- **Zero-Knowledge Blockchain Applications**

Examples: Zcash [21], Aztec [22]

Use proofs to enable private transactions, faster batch verification, and succinct blockchain state compression.



Image source: z.cash

- **Identity and Credential Verification**

Examples: World ID [23]

Prove attributes such as age, citizenship, or qualifications without revealing personal information.



Image source: <https://aztec.network/>

- **Verifiable Computation and Responsible Disclosure**

Example: Google Quantum AI [24, 25, 26]

Prove that a computation or scientific result was performed correctly without revealing the underlying algorithm, code, or data.

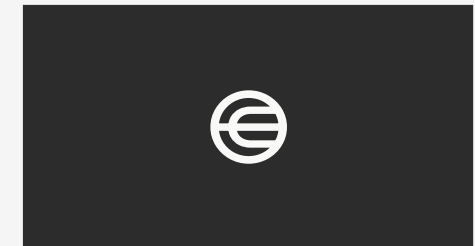


Image source: world.org/brand

[21] Zcash. Available from: <https://z.cash/>. [22] Aztec Protocol. Available from: <https://aztec.network/>. [23] World ID. Available from: <https://world.org/world-id>. [24] Google Quantum AI. Available from: <https://quantumai.google/>. [25] Google Research. Safeguarding Cryptocurrency by Disclosing Quantum Vulnerabilities Responsibly; 2026. Available from: <https://research.google/blog/safeguarding-cryptocurrency-by-disclosing-quantum-vulnerabilities-responsibly/>. [26] Babbush R, Zalcman A, Gidney C, Broughton M, Khattar T, Neven H, et al. Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations; 2026. arXiv preprint arXiv:2603.28846. Available from: <https://arxiv.org/abs/2603.28846>.

Researchers from The Trail of Bits broke Google's zero-knowledge proof of quantum cryptanalysis [27]

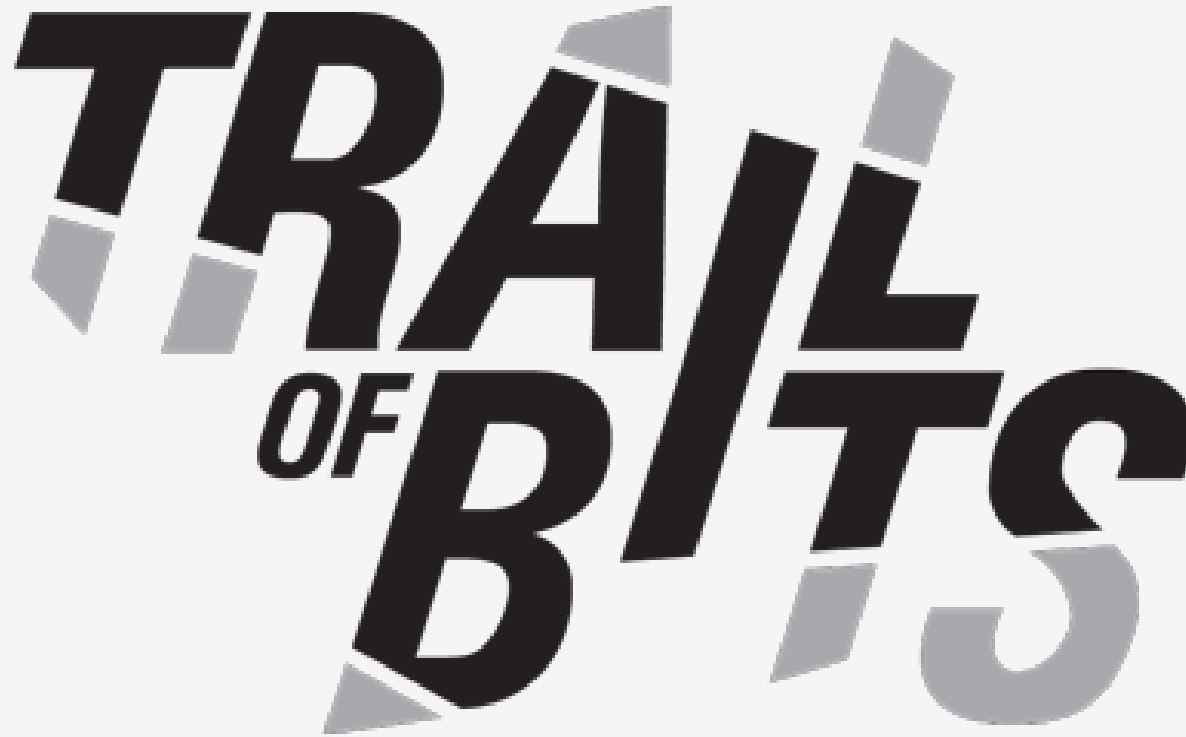


Image source: <https://trailofbits.com/>

[27] Ryan K. We beat Google's zero-knowledge proof of quantum cryptanalysis. Trail of Bits Blog; 2026. Available from: <https://blog.trailofbits.com/2026/04/17/we-beat-googles-zero-knowledge-proof-of-quantum-cryptanalysis/>.

It seems these systems could benefit from a little more testing...

In this project, we explore automated testing techniques for ZKP compilers.

02

Current Landscape & Enhancements

Existing Tools



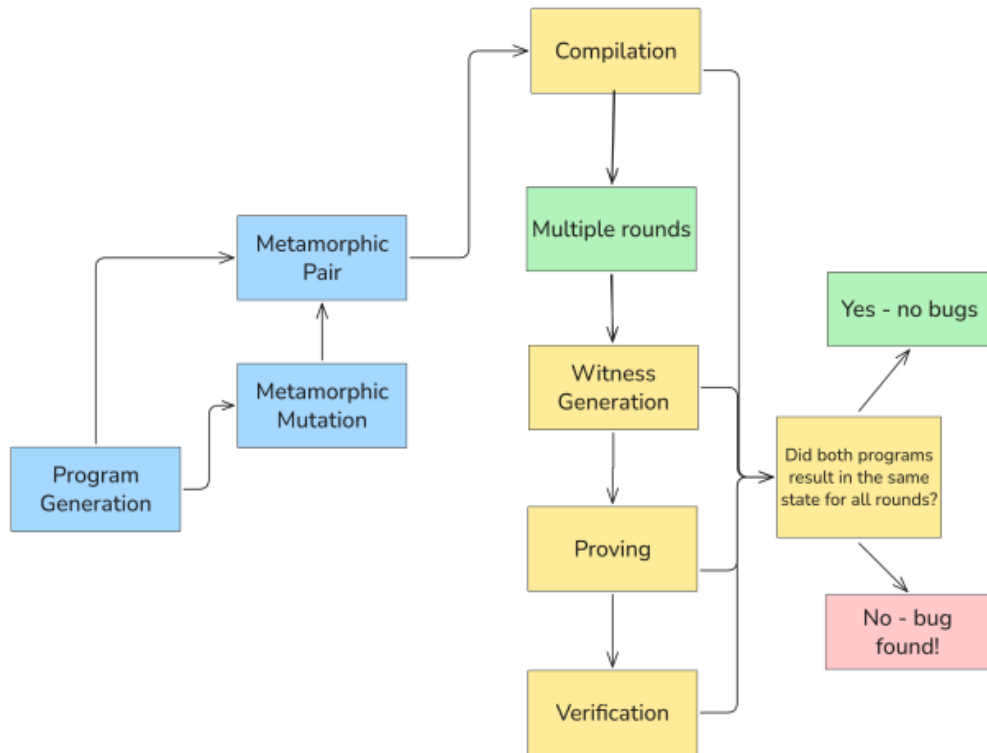
Circuzz ^[5]

MTZK ^[6]

[5] Hochrainer C, Isychev A, Wüstholtz V, Christakis M. Fuzzing Processing Pipelines for Zero-Knowledge Circuits. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25). ACM; 2025. p. 783-97. Available from: <https://doi.org/10.1145/3719027.3744791>.

[6] Xiao D, Liu Z, Peng Y, Wang S. Testing and Exploring Bugs in Zero-Knowledge (ZK) Compilers. In: Network and Distributed System Security Symposium (NDSS). Internet Society; 2025. Accessed: 2026-01-18. Available from: <https://doi.org/10.14722/ndss.2025.230530>.

Circuzz Architecture



Original expression



Mutated expression

↓ *via one of the metamorphic relations* ↓

comm-add: $(a + b) \leftrightarrow (b + a)$

dist-mul-add: $((a+b)*c) \leftrightarrow (a*c + b*c)$

double-negation: $!!a \leftrightarrow a$

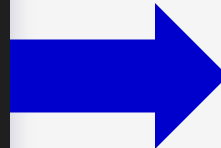
de-morgan: $!(a \ || \ b) \leftrightarrow (!a \ \&\& \ !b)$

relation rewrite: $(a \ != \ b) \leftrightarrow !(a \ == \ b)$

Enhancing Circuzz

- **Added support for 2 new DSLs: ZoKrates and o1js [8]**
- **Built a new constraint generator for Circom** — the original generator wrapped assertions in `assert()`, which don't contribute constraints to the R1CS, so the new one decomposes expressions directly into valid quadratic constraints instead

```
9  pragma circom 2.1.6;
8
7  template Main() {
6      signal input a;
5      signal input b;
4
3      assert(a * a * b - b == a + 42);
2  }
1
10 component main = Main();
```



```
template Main() {
    signal input a;
    signal input b;

    signal aa;
    aa <== a * a;

    signal aab;
    aab <== aa * b;

    signal lhs;
    signal rhs;

    lhs <== aab - b;
    rhs <== a + 42;

    component eq = IsEqual();

    eq.in[0] <== lhs;
    eq.in[1] <== rhs;

    eq.out == 1;
}
```

Circom: non-quadratic expression decomposed into valid R1CS constraints

[8] O(1) Labs. o1js: TypeScript Framework for zk-SNARKs and zkApps; 2023. Software repository. Accessed: 2026-01-10. Available from: <https://github.com/o1-labs/o1js>.

Enhancing Circuzz – Areas of Improvement

1. Highly random programs lacking "interesting" structure

Inspecting the generated programs reveals this directly, e.g. most satisfying assertions don't meaningfully connect to the inputs:

Backend	Connected SAT	Disconnected SAT
Circom	54.5%	45.5%
Gnark	56.9%	43.1%

Table 3.2: Structure of satisfying assertions in SAT circuits of a 30 min campaign.

2. No bugs found in the constraint generation stage

The original Circuzz paper doesn't report any such bugs.

Can we build a tool that analyses the constraints more thoroughly and generates more structured programs?

03

SMT Based Framework

What is an SMT Formula?

An SMT formula is a logical formula, **like in propositional logic, but extended with non-Boolean elements, predicates, and functions.**

$$(x - y > 2) \wedge (y = x + 7) \wedge (y < 5)$$

Theory

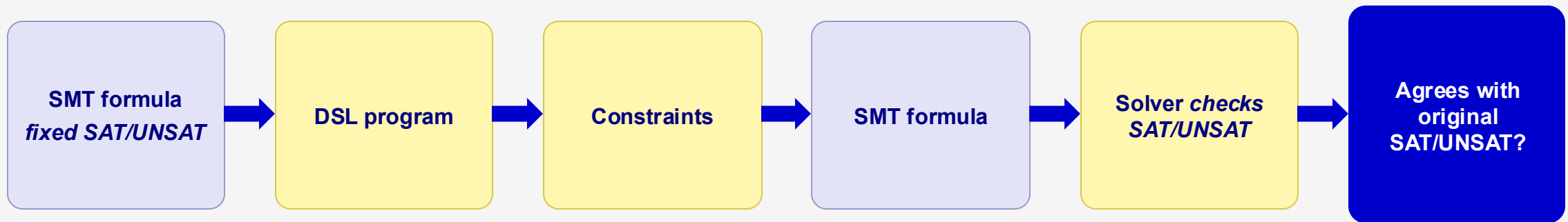
The set of these non-Boolean elements, predicates, and functions

e.g linear arithmetic, bit-vectors

Idea

Verify whether ZKP compilers preserve the semantics of SMT formulas through compilation.

- **Generate** structured SMT formulas with certain satisfiability
- **Translate** each formula into a ZKP DSL program and compile it into a constraint system
- **Check** the compiled constraints with an SMT solver used as a testing oracle

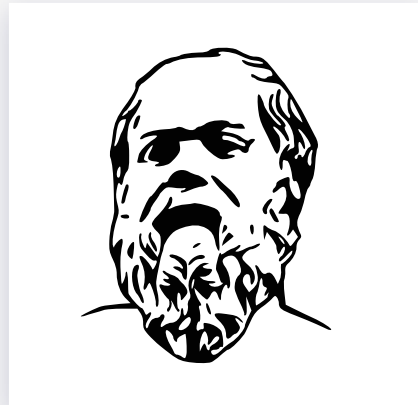


A mismatch between the two SAT/UNSAT results indicates a compiler bug

Supported DSLs

DSLs explored during this project, grouped by their constraint representation

R1CS-based



ZoKrates



Gnark

Other



Image sources: Circom: docs.circom.io · Gnark: github.com/Consensys/gnark · Noir: noir-lang.org · ZoKrates: <https://github.com/zokrates/zokrates>

Program Generation – Semantic Fusion

YinYang's [12] semantic fusion merges two seed formulas of the same satisfiability into one, **preserving satisfiability while creating diverse new test cases.**

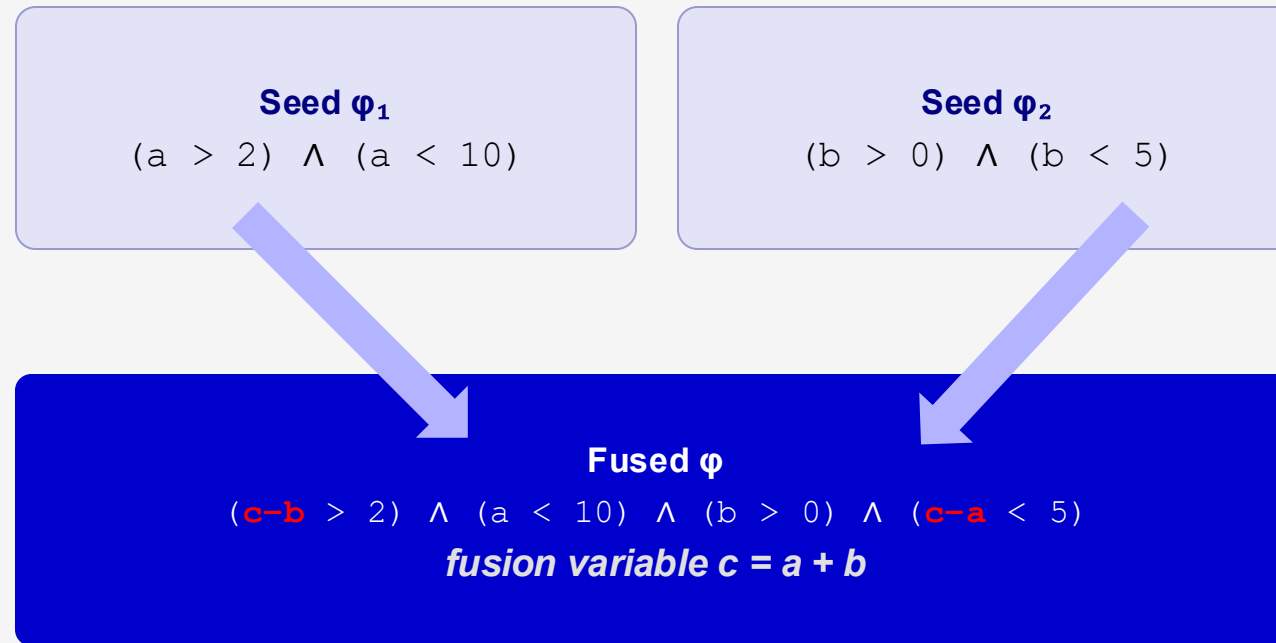


Image source:
github.com/testsm/yinyang

[12] Winterer D, Zhang C, Su Z. Validating SMT Solvers via Semantic Fusion. In: Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation; 2020. p. 718-30. Available from: <https://doi.org/10.1145/3385412.3385985>.

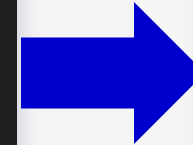
SMT Theories

Core

```
11 (set-logic QF_UF)
10
9  (declare-const a Bool)
8  (declare-const b Bool)
7  (declare-const c Bool)
6
5  (assert (or a b))
4  (assert (⇒ a c))
3  (assert (⇒ b (not c)))
2
1  (check-sat)
12 (get-model)
```

Integer

```
11 (set-logic QF_LIA)
10
9  (declare-const x Int)
8  (declare-const y Int)
7
6  (assert (≥ x 0))
5  (assert (≥ y 0))
4  (assert (= (+ (* 2 x) y) 10))
3  (assert (> x y))
2
1  (check-sat)
12 (get-model)
```



Finite Field

(CVC5 only [13, 14])

```
1 (set-logic QF_FF)
2 (define-sort F () (_ FiniteField 7))
3
4 (declare-const x F)
5 (declare-const y F)
6 (declare-const z F)
7
8 ; z = x * y
9 (assert (= z (ff.mul x y)))
10
11 ; x + y = 5
12 (assert (= (ff.add x y) (as ff5 F)))
13
14 ; z = 6
15 (assert (= z (as ff6 F)))
16
17 (check-sat)
18 (get-model)
```

- *Inequalities were too expensive for R1CS based system*
- *Still worked for Noir*

[13] Barbosa H, Barrett C, Brain M, Kremer G, Lachnitt H, Mann M, et al. cvc5: A Versatile and Industrial-Strength SMT Solver. In: Tools and Algorithms for the Construction and Analysis of Systems. vol. 13243 of Lecture Notes in Computer Science. Springer; 2022. p. 415-42. Available from: https://doi.org/10.1007/978-3-030-99524-9_24.

[14] Ozdemir A, Kremer G, Tinelli C, Barrett C. Satisfiability Modulo Finite Fields. In: Computer Aided Verification. Springer; 2023. p. 163-86. Available from: https://doi.org/10.1007/978-3-031-37703-7_8.

Getting Benchmarks for YinYang

Fusion needs seed formulas to start from.
These were sourced differently per theory

Boolean Seeds

- **Existing SAT benchmark suites [9]:** general satisfiable/unsatisfiable formulas for the standard oracle

Finite-Field Seeds

- **Boolean-to-field translation:** compiled existing Boolean benchmarks through ZK compilers into R1CS
- **Grammar-based generation:** used **ET [10]** with a custom grammar
- **Cryptography-inspired benchmarks:** recover the input to a Poseidon [11] permutation

[9] Hoos HH, Stütze T. SATLIB Benchmark Problems; 2000. Accessed: 2026-05-12. <https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html>.

[10] Winterer D, Zhang C, Su Z. Validating SMT Solvers for Correctness and Performance via Grammar-Based Enumeration. Proceedings of the ACM on Programming Languages. 2024;8(OOPSLA2):2378-401. Available from: <https://doi.org/10.1145/3689795>.

[11] Grassi L, Khovratovich D, Rechberger C, Roy A, Schafnegg M. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. In: 30th USENIX Security Symposium (USENIX Security 21). USENIX Association; 2021. p. 519-35. Available from: <https://eprint.iacr.org/2019/458>.

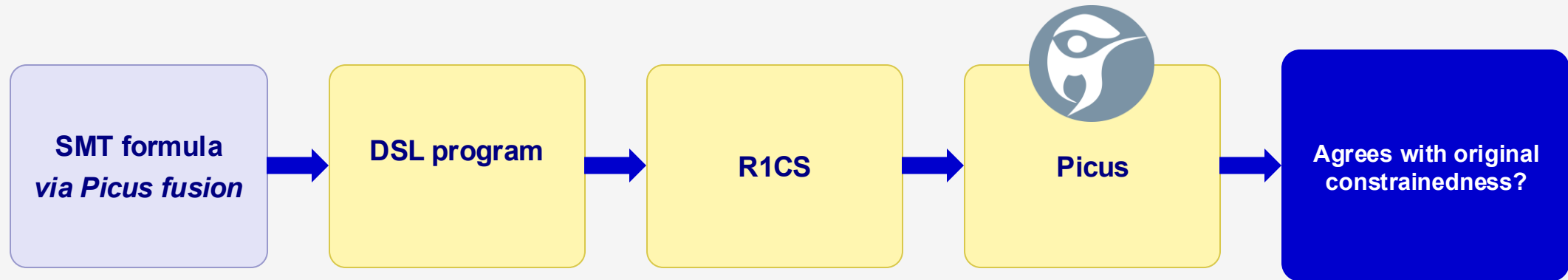
Picus Oracle

What is Picus [18, 28]?

- Static analysis tool for detecting **underconstrained R1CS circuits**
- Checks whether outputs are uniquely determined by the inputs

How I used it

- Modified semantic fusion to generate SMT formulas that are **constrained by construction**
- Used Picus as a **further oracle** for programs generated with the updated fusion method



If the compiled circuit is no longer fully constrained, the compiler has introduced a bug

[18] Pailoor S, Chen Y, Wang F, Rodríguez C, Van Geffen J, Morton J, et al. Automated Detection of Under-Constrained Circuits in Zero-Knowledge Proofs. Proceedings of the ACM on Programming Languages. 2023;7(PLDI):1510-32. Available from: <https://doi.org/10.1145/3591282>.

[28] Veridise. Picus: Automated Verification of Uniqueness Property for ZKP Circuits;. Software repository. Available from: <https://github.com/Veridise/Picus>.

04

Results & Evaluation

Answering the Question

Can we build a tool that analyses the constraints more thoroughly and generates more structured programs?

Yes. This approach does both:

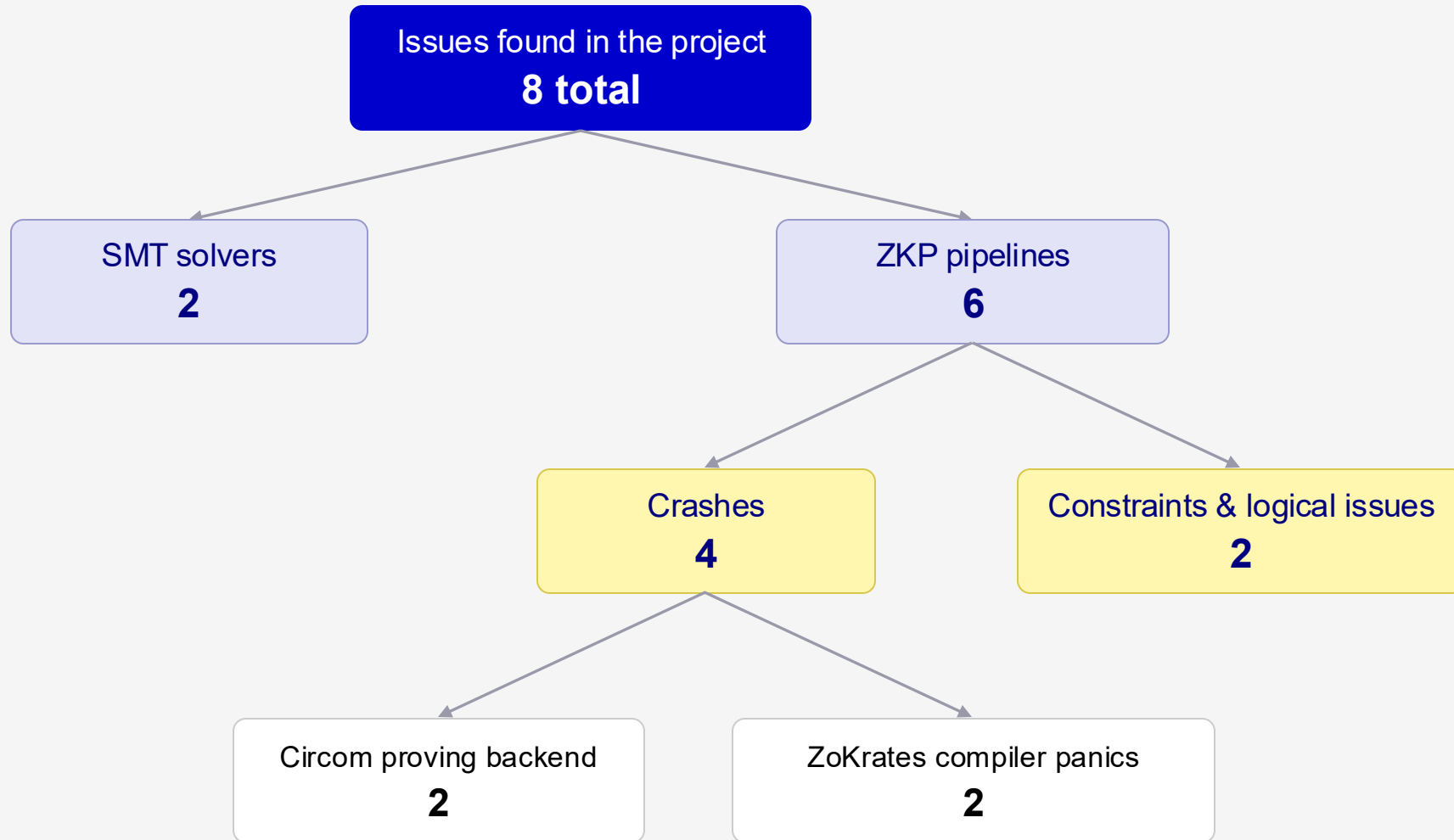
Generates more structured programs

- Programs are generated from **seed SMT formulas**, rather than being produced purely at random.

Analyses constraints more thoroughly

- The tool **inspects the generated constraints by running solvers on them.**

Issues Found



Example Issue

Negative integers are treated as field elements before the cast

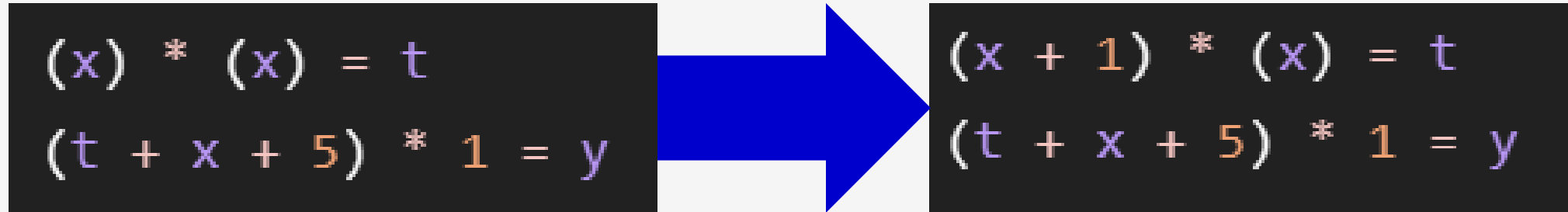
Issue: <https://github.com/noir-lang/noir/issues/12794>

```
fn main() {  
    assert(  
        -(1 as i64) == -1  
    ); // WORKS  
    assert(  
        (-1 as i64) == -1  
    ); // FAILS  
}
```

Artificial Bugs

Real logical bugs in constraint generation bugs seem to be rare.

To evaluate the testing approach, I used additional techniques, including **artificial bugs injection**.



Configuration	Bugs Found	Bugs / Program	Bugs / Second
SMT SAT	5/5	0.2273	0.0166
SMT + Picus	5/5	0.2273	0.0104
SMT UNSAT	0/5	0.0000	0.0000
Circuzz Standard	2/5	0.0088	0.0014
Circuzz + Picus	0/5	0.0000	0.0000

Table 5.2: Aggregate results of the artificial bug injection experiments.

06

Conclusion

Key Contributions

- **Extended Circuzz** so generated Circom programs test constraint generation, and added support for additional ZKP languages.
- **Designed and implemented an SMT-based generator** that produces structured seed programs.
- **Designed and implemented a constraint-inspection oracle** that directly analyses generated constraints to detect semantic correctness bugs
- **Generated finite-field SMT benchmarks** derived from ZKP circuits and other sources, providing new workloads for SMT solvers in a domain with very few public benchmarks.
- **Released the implementation and experimental infrastructure** developed during the project as open-source software.

Other Work

This presentation does not cover every technical challenge I investigated. Other work included:

- **Constraint-cluster inflation:** triggering Circom's large-cluster optimisation with synthetic negation chains.
- **Adaptive hints:** using partial solutions to reduce solver timeouts.
- **Integer theories in Noir:** revisiting integer generation to test casts and overflows.
- **Array theories:** adding support for SMT arrays and bounds checks.
- **Alternative generators:** exploring tools such as **smtfuzz** [20].

[20] Yao P, Huang H, Tang W, Shi Q, Wu R, Zhang C. Fuzzing SMT Solvers via Two-Dimensional Input Space Exploration. In: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '21). ACM; 2021. p. 322-35. Available from: <https://doi.org/10.1145/3460319.3464803>.

Further Work

- **Go deeper on Noir and ACIR**

Richer language features and compiler transformations make Noir a promising target for semantic bugs.

- **Explore more SMT theories**

String and regex support, e.g. zkRegex [29], could be tested using string theories.

- **Broaden testing objectives**

The same SMT-generated programs could target more compiler correctness issues beyond constraint generation.

- **Add complementary semantic oracles**

Tools such as zkFuzz [30] could analyse the same benchmarks from a different testing perspective.

[29] Gupta A, Londhe S, Bisht A, Panda S, Suegami S. ZK Regex. Zenodo; 2025. Software; archived via GitHub repository, original 2022. Available from: <https://doi.org/10.5281/zenodo.15603470>.

[30] Takahashi H, Kim J, Jana S, Yang J. zkFuzz: Foundation and Framework for Effective Fuzzing of Zero-Knowledge Circuits; 2025. Accessed: 2026-01-18. arXiv preprint arXiv:2504.11961. Available from: <https://arxiv.org/abs/2504.11961>.

IMPERIAL

Thank you

Further Topics

Inflating Linear Clusters

- **Problem:** Circom's large-cluster optimisation pass rarely trigger because generated linear constraint clusters stay below its activation threshold
- **Solution:** inject synthetic *negation chains*.
- Default chain length is **400 constraints**, which is enough to cross the optimisation threshold
- The chains don't change the satisfiability of the formula
- Synthetic variables and constraints are stripped from the R1CS after compilation so the solver doesn't do any additional work.

```
x2 ← 1 - x1;  
x3 ← 1 - x2;  
x4 ← 1 - x3;  
x5 ← 1 - x4;  
x6 ← 1 - x5;  
x7 ← 1 - x6;  
x8 ← 1 - x7;  
x9 ← 1 - x8;  
x10 ← 1 - x9;  
// ...  
x399 ← 1 - x398;  
x400 ← 1 - x399;
```

Adaptive Hints

- **Problem:** compiled constraint systems can be large, so the solver often spends most of its time searching
- **Idea:** reuse the satisfying assignment of the original SMT formula and inject it as partial wire hints before solving the compiled R1CS
- Each wire is hinted independently with same probability
- **Adaptive:** more timeouts → more hints; stable solving → fewer hints, more extracted models
- **Trade-off:** fixing intermediate values can reduce the solver's ability to explore alternative satisfying assignments

Results:

1.8×

*faster mean solve time on 15-
variable benchmarks*

15 → 21 variables

*furthest benchmark size solved
before 30 sec timeout*

Integer & Array Theories for Noir

- Re-visited integer theories for Noir
- Exercised the casting mechanism
- Added support for array theories
- Needed to augment the SMT queries with bound checks for overflows
- Also explored alternatives to **YinYang**, such as **smtfuzz** [20]

[20] Yao P, Huang H, Tang W, Shi Q, Wu R, Zhang C. Fuzzing SMT Solvers via Two-Dimensional Input Space Exploration. In: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '21). ACM; 2021. p. 322-35. Available from: <https://doi.org/10.1145/3460319.3464803>.

```
20 mod safe_math;
19
18 pub struct G1 {
17   12 : u16,
16   v3 : bool,
15   v6 : bool,
14 }
13
12 pub fn main(Bool0_0 : [bool; 3], g1 : G1, v1 : bool, _arr_601051533455607942_601051533455607942_0 : [164; 31], v2 : bool,
11   _arr_601051533455607942_601051533455607942_1 : [164; 31], _arr_601051533455607942_4407170993114260732_0 : [bool; 8], _arr_601051533455607942_1513244341096956722_0
10   : [[bool; 8]; 8], i5 : u16) {
9   assert((g1.12 <= 32767), "signed_cap");
8   assert((i5 <= 32767), "signed_cap");
7   assert(((127 as i64) >= (-15_i64)) | false), "assert_3");
6   let mid_0 : bool = (v1 | v2);
5   let mut store_1 : [164; 31] = arr_601051533455607942_601051533455607942_1;
4   store_1[(30 as u32)] = (74 as i64);
3   let mid_2 : i64 = (((((51 as u16) + (87 as u16)) as u32) + (30 as u32)) as i64) + store_1[(0 as u32)]);
2   let mid_3 : bool = ((g1.12 as i64) < mid_2);
1   let mid_4 : u16 = ((51 as u16) + (87 as u16));
0   let mut store_5 : [164; 31] = arr_601051533455607942_601051533455607942_1;
-1   store_5[(30 as u32)] = (74 as i64);
-2   let mid_6 : bool = (((mid_4 as u32) + (30 as u32)) as i64) + store_5[(0 as u32)] < (15 as i64);
-3   let mid_7 : bool = (mid_3 & mid_6);
-4   let mid_8 : i64 = (((-15_i64) - (15 as i64)));
-5   let mid_9 : bool = ((mid_0 | (mid_7 & ((398 as u32) + (398 as u32)) + (15 as u32)) + (119 as u32))) | (((74 as i64) = safe_math::abs_164
-6   ((((-15_i64) - (15 as i64)) - (74 as i64)) - (51 as i64)))) | (!((74 as i64) = safe_math::abs_164(((mid_8 - (74 as i64)) - (51 as i64))))));
-7   let mid_10 : bool = (mid_9 | bools_0[1]);
-8   assert((mid_10 | false), "assert_5");
-9   assert(((127 as i64) >= (-15_i64)) | false), "assert_8");
-10  let mid_11 : bool = ((v1 | v2);
-11  assert((((119 as u16) >= (15 as u16)) & ((15 as i64) >= (-15_i64)) | mid_11 | false), "assert_20");
-12 }
```

Noir test program exercising casts and array indexing

```
(assert (>= (+ 398 398) 0))
(assert (<= (+ 398 398) 65535))
(assert (>= (+ (+ 398 398) 15) 0))
(assert (<= (+ (+ 398 398) 15) 65535))
(assert (>= (+ (+ (+ 398 398) 15) 119) 0))
(assert (<= (+ (+ (+ 398 398) 15) 119) 4294967295))
(assert (>= 12 0))
(assert (<= 12 32767))
(assert (>= (+ 51 87) 0))
(assert (<= (+ 51 87) 65535))
(assert (>= (+ 51 87) 30) 0))
(assert (<= (+ 51 87) 30) 65535))
(assert (>= 15 0))
(assert (<= 15 30))
(assert (>= (- 15) 0))
(assert (<= (- 15) 30))
(assert (>= (+ (+ 51 87) 30) (select (store arr_601051533455607942_601051533455607942-1 15 74) (- 15))))
-9223372036854775808))
(assert (<= (+ (+ 51 87) 30) (select (store arr_601051533455607942_601051533455607942-1 15 74) (- 15)))) 9223372036854775807))
(assert (<= (- (- 15) 15) -9223372036854775808))
(assert (<= (- (- 15) 15) 9223372036854775807))
(assert (>= (- (- (- 15) 15) 74) -9223372036854775808))
(assert (<= (- (- (- 15) 15) 74) 9223372036854775807))
(assert (>= (- (- (- 15) 15) 74) 51) -9223372036854775808))
(assert (<= (- (- (- 15) 15) 74) 51) 9223372036854775807))
(assert (>= (- (- (- (- 15) 15) 74) 51) -9223372036854775808))
(assert (<= (- (- (- (- 15) 15) 74) 51) 9223372036854775807))
(assert (>= (ite (>= (- (- (- (- 15) 15) 74) 51) 0) (- (- (- (- 15) 15) 74) 51) (- (- (- (- 15) 15) 74) 51))))
-9223372036854775808))
(assert (<= (ite (>= (- (- (- (- 15) 15) 74) 51) 0) (- (- (- (- 15) 15) 74) 51) (- (- (- (- 15) 15) 74) 51))))
9223372036854775807))
(check-sat)
(get-model)
> Execute | ■ Stop
```

SMT assertions with overflow bound checks

Architecture Diagram

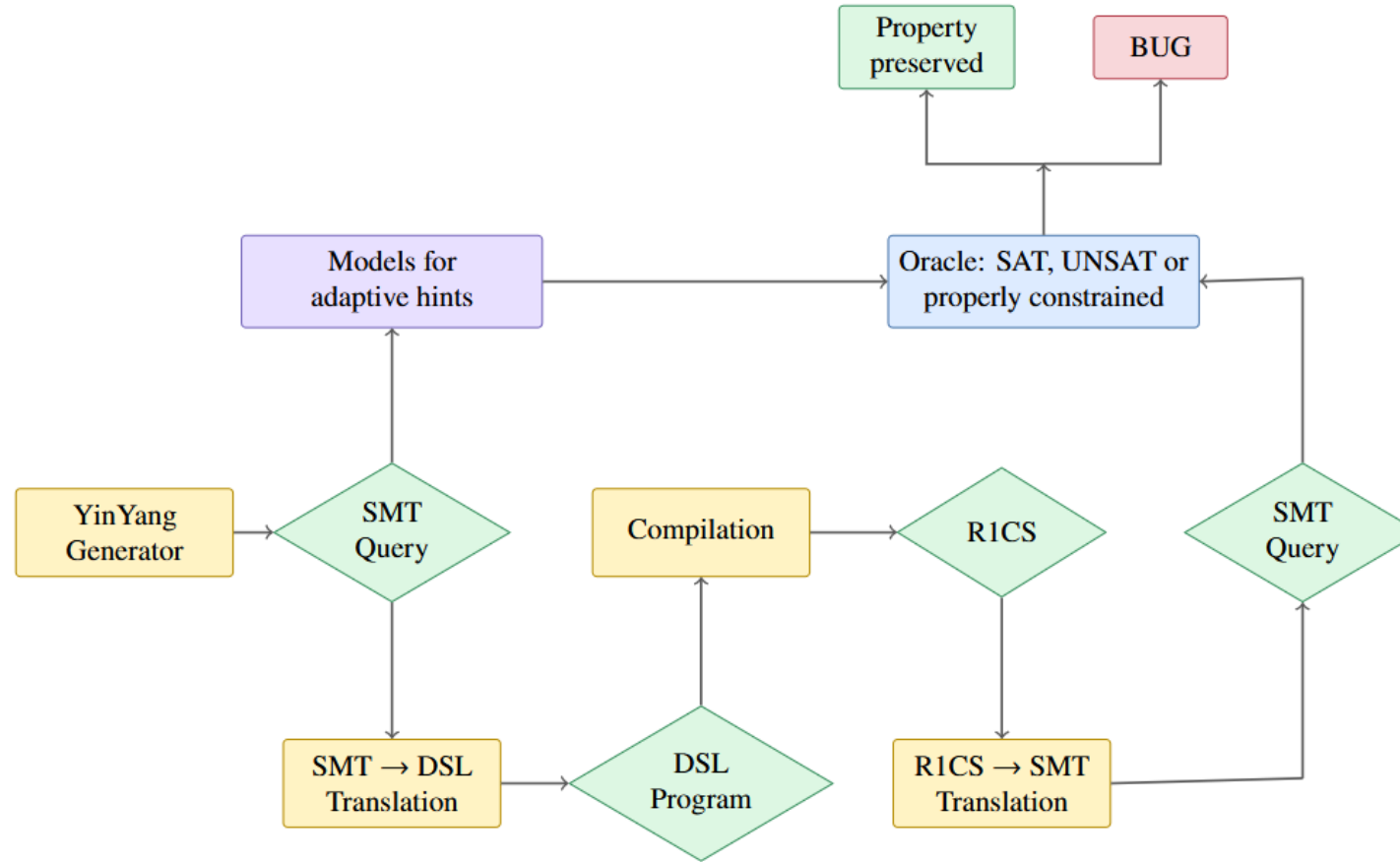


Figure 4.1: High-level architecture of the SMT-based testing approach.

References

References

- [1] Ozdemir A, Wahby RS, Brown F, Barrett CW. Bounded Verification for Finite-Field-Blasting in a Compiler for Zero Knowledge Proofs. In: Computer Aided Verification. Springer; 2023. p. 154-75. Accessed: 2026-01-18. Available from: https://doi.org/10.1007/978-3-031-37709-9_8.
- [2] Groth J. On the Size of Pairing-Based Non-Interactive Arguments. In: Advances in Cryptology – EUROCRYPT 2016. vol. 9666 of Lecture Notes in Computer Science. Springer; 2016. p. 305-26. Available from: https://doi.org/10.1007/978-3-662-49896-5_11.
- [3] Gabizon A, Williamson ZJ, Ciobotaru O. PLONK: Permutations over Lagrange-Bases for Oecumenical Noninteractive Arguments of Knowledge; 2019. Accessed: 2026-01-18. Cryptology ePrint Archive, Report 2019/953. Available from: <https://eprint.iacr.org/2019/953>.
- [4] Ben-Sasson E, Bentov I, Horesh Y, Riabzev M. Scalable, Transparent, and Post-Quantum Secure Computational Integrity; 2018. Accessed: 2026-01-18. Cryptology ePrint Archive, Report 2018/046. Available from: <https://eprint.iacr.org/2018/046>.
- [5] Hochrainer C, Isychev A, Wüstholtz V, Christakis M. Fuzzing Processing Pipelines for Zero-Knowledge Circuits. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25). ACM; 2025. p. 783-97. Available from: <https://doi.org/10.1145/3719027.3744791>.
- [6] Xiao D, Liu Z, Peng Y, Wang S. Testing and Exploring Bugs in Zero-Knowledge (ZK) Compilers. In: Network and Distributed System Security Symposium (NDSS). Internet Society; 2025. Accessed: 2026-01-18. Available from: <https://doi.org/10.14722/ndss.2025.230530>.
- [7] Eberhardt J, Tai F. ZoKrates: A Toolbox for zkSNARKs on Ethereum; 2018. Accessed: 2026-01-13. Available from: <https://zokrates.github.io/>.
- [8] O(1) Labs. o1js: TypeScript Framework for zk-SNARKs and zkApps; 2023. Software repository. Accessed: 2026-01-10. Available from: <https://github.com/o1-labs/o1js>.
- [9] Hoos HH, Stütze T. SATLIB Benchmark Problems; 2000. Accessed: 2026-05-12. <https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html>.
- [10] Winterer D, Zhang C, Su Z. Validating SMT Solvers for Correctness and Performance via Grammar-Based Enumeration. Proceedings of the ACM on Programming Languages. 2024;8(OOPSLA2):2378-401. Available from: <https://doi.org/10.1145/3689795>.
- [11] Grassi L, Khovratovich D, Rechberger C, Roy A, Schofnegger M. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. In: 30th USENIX Security Symposium (USENIX Security 21). USENIX Association; 2021. p. 519-35. Available from: <https://eprint.iacr.org/2019/458>.
- [12] Winterer D, Zhang C, Su Z. Validating SMT Solvers via Semantic Fusion. In: Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation; 2020. p. 718-30. Available from: <https://doi.org/10.1145/3385412.3385985>.
- [13] Barbosa H, Barrett C, Brain M, Kremer G, Lachnitt H, Mann M, et al. cvc5: A Versatile and Industrial-Strength SMT Solver. In: Tools and Algorithms for the Construction and Analysis of Systems. vol. 13243 of Lecture Notes in Computer Science. Springer; 2022. p. 415-42. Available from: https://doi.org/10.1007/978-3-030-99524-9_24.
- [14] Ozdemir A, Kremer G, Tinelli C, Barrett C. Satisfiability Modulo Finite Fields. In: Computer Aided Verification. Springer; 2023. p. 163-86. Available from: https://doi.org/10.1007/978-3-031-37703-7_8.
- [15] Bellés-Muñoz M, Isabel M, Muñoz-Tapia JL, Rubio A, Baylina Melé J. Circom: A Circuit Description Language for Building Zero-Knowledge Applications. IEEE Transactions on Dependable and Secure Computing. 2023;20(6):4733-51. Available from: <https://doi.org/10.1109/TDSC.2022.3232813>.
- [16] Noir Team. Noir;. Accessed: 2026-01-18. Available from: <https://noir-lang.org>.
- [17] ConsenSys. gnark Documentation;. Accessed: 2026-01-18. Available from: <https://docs.gnark.consensys.io>.
- [18] Pailoor S, Chen Y, Wang F, Rodríguez C, Van Geffen J, Morton J, et al. Automated Detection of Under-Constrained Circuits in Zero-Knowledge Proofs. Proceedings of the ACM on Programming Languages. 2023;7(PLDI):1510-32. Available from: <https://doi.org/10.1145/3591282>.
- [19] Aztec Protocol. ACIR: Abstract Circuit Intermediate Representation;. Software repository. Accessed: 2026-05-22. <https://github.com/noir-lang/acvm>.
- [20] Yao P, Huang H, Tang W, Shi Q, Wu R, Zhang C. Fuzzing SMT Solvers via Two-Dimensional Input Space Exploration. In: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '21). ACM; 2021. p. 322-35. Available from: <https://doi.org/10.1145/3460319.3464803>.

References

- [21] Zcash. Available from: <https://z.cash/>.
- [22] Aztec Protocol. Available from: <https://aztec.network/>.
- [23] World ID. Available from: <https://world.org/world-id>.
- [24] Google Quantum AI. Available from: <https://quantumai.google/>.
- [25] Google Research. Safeguarding Cryptocurrency by Disclosing Quantum Vulnerabilities Responsibly; 2026. Available from: <https://research.google/blog/safeguarding-cryptocurrency-by-disclosing-quantum-vulnerabilities-responsibly/>.
- [26] Babbush R, Zalcman A, Gidney C, Broughton M, Khatyar T, Neven H, et al. Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations; 2026. arXiv preprint arXiv:2603.28846. Available from: <https://arxiv.org/abs/2603.28846>.
- [27] Ryan K. We beat Google's zero-knowledge proof of quantum cryptanalysis. Trail of Bits Blog; 2026. Available from: <https://blog.trailofbits.com/2026/04/17/we-beat-googles-zero-knowledge-proof-of-quantum-cryptanalysis/>.
- [28] Veridise. Picus: Automated Verification of Uniqueness Property for ZKP Circuits;. Software repository. Available from: <https://github.com/Veridise/Picus>.
- [29] Gupta A, Londhe S, Bisht A, Panda S, Suegami S. ZK Regex. Zenodo; 2025. Software; archived via GitHub repository, original 2022. Available from: <https://doi.org/10.5281/zenodo.15603470>.
- [30] Takahashi H, Kim J, Jana S, Yang J. zkFuzz: Foundation and Framework for Effective Fuzzing of Zero-Knowledge Circuits; 2025. Accessed: 2026-01-18. arXiv preprint arXiv:2504.11961. Available from: <https://arxiv.org/abs/2504.11961>.